

VNUHCM - UNIVERSITY OF SCIENCE

Faculty of Information Technology



String Matching Project Report

CSC10004 Data Structures & Algorithms – Spring 2026

Group Information:

Student Name	Student ID	Assigned Tasks	Completed (%)
Huỳnh Phúc Nguyên	25127106	Sub-Implementation, Project Organization	100
Đỗ Đăng Tú	25127167	Main-Implementation	100
Đỗ Minh Khang	25127363	Report Contents (Problem Statement, Algorithm Analysis)	100
Huỳnh Đình Khương	25127396	Experiments, Visualization, Commentary	100
Total Group Completion			100

Contents

1. Problem Statement.....	2
2. Algorithms Analysis.....	2
2.1. Naïve Approach (Brute-Force)	2
2.2. Rabin-Karp Algorithm	5
2.3. Knuth-Morris-Pratt (KMP) Algorithm.....	9
2.4. Boyer-Moore Algorithm	13
2.5. Aho-Corasick Algorithm.....	19
3. Experiment.....	26
3.1. Context.....	26
3.2. Results.....	27
4. References.....	31
5. Acknowledgment	32

1. Problem Statement

The string-matching problem is a foundational problem in computer science defined as follows. Given a **text** T of length n and a **pattern** P of length m , where both T and P are strings composed of characters drawn from a **finite alphabet** Σ , the objective is to identify all valid shifts s such that the substring of T beginning at position s and ending at position $s + m - 1$ is identical to P . Formally, a shift s is considered valid if and only if:

$$T[s], T[s+1], \dots, T[s+m-1] = P[0], P[1], \dots, P[m-1]$$

where $0 \leq s \leq n - m$.^[1]

In the context of this project, the string-matching problem is applied directly to a one-dimensional sequence of characters. The text T is defined as a string of length n , and the pattern P is a keyword of length m provided in the input. The objective is to identify all positions within T at which P occurs as a contiguous substring. All five algorithms under investigation are applied to this formulation, and their correctness and performance are evaluated on the basis of their behaviour over such linear character sequences. The extension of this problem to two-dimensional structures, such as a character grid, is addressed separately in the experimental section of this report.

2. Algorithms Analysis

2.1. Naïve Approach (Brute-Force)

2.1.1. Introduction and Application

The Naïve algorithm is the most direct solution to the string-matching problem. It slides the pattern P over the text T one position at a time, performing a left-to-right character-by-character comparison at each candidate shift. If all m characters match, a valid occurrence is reported; if any mismatch is encountered, the current alignment is abandoned and the pattern advances by one position. No preprocessing phase is required, making it immediately applicable without any prior computation.^[1]

Despite its simplicity, the Naïve algorithm retains practical value when the pattern changes frequently, when a search is performed only once, or when memory constraints prohibit the use of auxiliary data structures. It also serves as the standard performance baseline against which all optimised algorithms are measured.^[1]

2.1.2. Pseudocode

```
NAIVE-STRING-MATCHER(T, P):  
    n = length of T  
    m = length of P  
    for s = 0 to (n - m):  
        match = true  
        for i = 0 to (m - 1):  
            if T[s + i] != P[i]:  
                match = false  
                break  
        if match == true:  
            print "Pattern occurs at shift s"
```

2.1.3. Step-by-Step Trace

Using $T = \text{"abacabab"}$ ($n = 8$) and $P = \text{"abab"}$ ($m = 4$), the algorithm evaluates shifts $s = 0$ through $s = 4$ (since $n - m = 4$).

Shift $s = 0$ – Aligning P at $T[0..3]$:

- Text window: a b a c

- Pattern P : a b a b

- $i = 0$: $T[0] = \text{'a'}$ vs $P[0] = \text{'a'} \rightarrow \text{Match}$
- $i = 1$: $T[1] = \text{'b'}$ vs $P[1] = \text{'b'} \rightarrow \text{Match}$
- $i = 2$: $T[2] = \text{'a'}$ vs $P[2] = \text{'a'} \rightarrow \text{Match}$
- $i = 3$: $T[3] = \text{'c'}$ vs $P[3] = \text{'b'} \rightarrow \textbf{Mismatch}$ – inner loop terminates

- Comparisons: 4.

Shift $s = 1$ – Aligning P at $T[1..4]$:

- Text window: b a c a

- Pattern P : a b a b

- $i = 0$: $T[1] = \text{'b'}$ vs $P[0] = \text{'a'} \rightarrow \textbf{Mismatch}$ – inner loop terminates.

- Comparisons: 1.

Shift s = 2 – Aligning P at T[2..5]:

- Text window: a c a b

- Pattern P: a b a b

- $i = 0$: $T[2] = 'a'$ vs $P[0] = 'a' \rightarrow$ Match
- $i = 1$: $T[3] = 'c'$ vs $P[1] = 'b' \rightarrow$ **Mismatch** – inner loop terminates.

- Comparisons: 2.

Shift s = 3 – Aligning P at T[3..6]:

- Text window: a c a b

- Pattern P: a b a b

- $i = 0$: $T[3] = 'c'$ vs $P[0] = 'a' \rightarrow$ **Mismatch** – inner loop terminates immediately.

- Comparisons: 1.

Shift s = 4 – Aligning P at T[4..7]:

- Text window: a b a b

- Pattern P: a b a b

- $i = 0$: $T[4] = 'a'$ vs $P[0] = 'a' \rightarrow$ Match
- $i = 1$: $T[5] = 'b'$ vs $P[1] = 'b' \rightarrow$ Match
- $i = 2$: $T[6] = 'a'$ vs $P[2] = 'a' \rightarrow$ Match
- $i = 3$: $T[7] = 'b'$ vs $P[3] = 'b' \rightarrow$ Match

- Comparisons: 4. **A valid occurrence is reported at shift s = 4.**

Summary:

Shift s	Text window	Comparisons	Outcome
0	a b a c	4	Mismatch at i=3
1	b a c a	1	Mismatch at i=0
2	a c a b	2	Mismatch at i=1
3	c a b a	1	Mismatch at i=0
4	a b a b	4	Match found at s=4
Total		12	

2.1.4. Complexity

Scenario	Time Complexity	Space Complexity
Best Case	$O(n)$	$O(1)$
Worst Case	$O((n - m + 1) * m)$	$O(1)$
Average Case (Large $ \Sigma $)	$O(n)$	$O(1)$

No preprocessing is required. The space complexity is $O(1)$ as the algorithm operates entirely in-place using two index variables.^[1]

2.1.5. Discussion

The Naïve algorithm performs well when the alphabet is large, because characters drawn from a large alphabet are statistically unlikely to match at random positions, causing the inner loop to exit after the very first comparison in most cases. Under these conditions, its average-case performance approaches $O(n)$, making it competitive with more sophisticated algorithms.^[1]

Conversely, the algorithm degrades severely when the text and pattern share a high degree of repetition over a small alphabet. The canonical worst-case example is $T = \text{"aaaa...aab"}$ (n characters) and $P = \text{"aab"}$ (m characters). At each of the $n - m + 1$ candidate shifts, the algorithm performs $m - 1$ successful comparisons before encountering the mismatch on the final character, yielding a total of $\Theta(n * m)$ comparisons. In the context of this project, this worst-case scenario can arise when searching for a pattern within a highly uniform character grid (e.g., a grid composed predominantly of a single repeated character).^[1]

2.2. Rabin-Karp Algorithm

2.2.1. Introduction and Application

The Rabin-Karp algorithm, introduced by Michael O. Rabin and Richard M. Karp in 1987, addresses a fundamental inefficiency of the Naïve approach: the redundant re-examination of characters within overlapping windows. Rather than comparing the pattern against each substring of the text character by character, Rabin-Karp computes a numerical **hash value** for the pattern P and a corresponding hash value for each length- m window of T . A full character-by-character comparison is performed only when the two hash values are equal, an event referred to as a **spurious hit** if the strings do not actually match. When a high-quality hash function is used, spurious hits are rare, and the algorithm's expected running time approaches $O(n + m)$.^[2]

The hash function employed by Rabin-Karp is based on polynomial rolling hashing, also known as the Rabin fingerprint. A string is treated as a polynomial evaluated at a base value d (typically the size of the alphabet), with arithmetic performed modulo a large prime q to prevent integer overflow. The key efficiency gain lies in the ability to compute the hash value for window $T[s+1 .. s+m]$ from the hash value for $T[s .. s+m-1]$ in $O(1)$ time using the following recurrence, known as the **rolling hash formula**:

$$t_hash(s+1) = (d * (t_hash(s) - T[s] * h) + T[s+m]) \bmod q$$

where $h = d^{(m-1)} \bmod q$ represents the weight of the most significant character in the window. [\[2\]](#)

Its primary weakness lies in the risk of spurious hits – cases where a window's hash matches p_hash but the actual characters do not, forcing an unnecessary verification. If such collisions occur frequently due to a poorly chosen prime modulus, performance degrades to $O(nm)$. For single-pattern search where worst-case guarantees are required, KMP or Boyer-Moore are generally preferable. [\[2\]](#) [\[8\]](#)

2.2.2. Pseudocode

```
RABIN-KARP-MATCHER(T, P, d, q):
    // d = alphabet size
    // q = a chosen prime modulus

    n = length of T
    m = length of P
    h = d^(m-1) mod q
    p_hash = 0
    t_hash = 0

    for i = 0 to (m - 1):
        p_hash = (d * p_hash + P[i]) mod q
        t_hash = (d * t_hash + T[i]) mod q

    for s = 0 to (n - m):
        if p_hash == t_hash:
            if T[s .. s+m-1] == P[0 .. m-1]:
                print "Pattern found at shift s"

        if s < n - m:
            t_hash = (d * (t_hash - T[s] * h) + T[s + m]) mod q
            if t_hash < 0:
                t_hash = t_hash + q
```

2.2.3. Step-by-Step Trace

Using $T = \text{"abacabab"} (n = 8)$ and $P = \text{"abab"} (m = 4)$ with the parameters $d = 10$ (simplified base) and $q = 13$ (prime modulus). Characters are encoded as integers: $a = 1, b = 2, c = 3$.

Preprocessing Phase – Computing h and Initial Hash Values:

The high-order weight h is computed as:

$$h = d^{(m-1)} \bmod q = 10^3 \bmod 13 = 1000 \bmod 13 = 12$$

The pattern hash p_hash is computed using Horner's method:

$$\begin{aligned} p_hash &= (((1 * 10 + 2) * 10 + 1) * 10 + 2) \bmod 13 \\ &= (((12) * 10 + 1) * 10 + 2) \bmod 13 \\ &= ((121) * 10 + 2) \bmod 13 \\ &= 1212 \bmod 13 \\ &= 3 \end{aligned}$$

Search Phase – Rolling Hash and Window Comparison:

The initial text window hash $t_hash(0)$ for $T[0..3] = \text{"abac"}$ is computed as:

$$\begin{aligned} t_hash(0) &= (((1 * 10 + 2) * 10 + 1) * 10 + 3) \bmod 13 \\ &= 1213 \bmod 13 \\ &= 4 \end{aligned}$$

$t_hash(1)$ for window "baca" (removing $T[0]='a'=1$, adding $T[4]='a'=1$):

$$\begin{aligned} t_hash(1) &= (10 * (4 - 1 * 12) + 1) \bmod 13 \\ &= (10 * (-8) + 1) \bmod 13 \\ &= (-79) \bmod 13 \\ &= (-79 + 7 * 13) \bmod 13 = 12 \end{aligned}$$

$t_hash(2)$ for window "acab" (removing $T[1]='b'=2$, adding $T[5]='b'=2$):

$$\begin{aligned} t_hash(2) &= (10 * (12 - 2 * 12) + 2) \bmod 13 \\ &= (10 * (-12) + 2) \bmod 13 \\ &= (-118) \bmod 13 \\ &= (-118 + 10 * 13) \bmod 13 = 12 \end{aligned}$$

t_hash(3) for window "caba" (removing T[2]='a'=1, adding T[6]='a'=1):

$$\begin{aligned} t_hash(3) &= (10 * (12 - 1 * 12) + 1) \bmod 13 \\ &= (10 * 0 + 1) \bmod 13 \\ &= 1 \end{aligned}$$

t_hash(4) for window "abab" (removing T[3]='c'=3, adding T[7]='b'=2):

$$\begin{aligned} t_hash(4) &= (10 * (1 - 3 * 12) + 2) \bmod 13 \\ &= (10 * (-35) + 2) \bmod 13 \\ &= (-348) \bmod 13 \\ &= (-348 + 27 * 13) \bmod 13 = 3 \end{aligned}$$

At shift $s = 4$, $t_hash(4) = 3 = p_hash$. A character-by-character verification is triggered: $T[4..7] = \text{"abab"}$ matches $P[0..3] = \text{"abab"}$. **A valid occurrence is confirmed at shift $s = 4$.** No spurious hits occurred in this example, as no two distinct windows produced the same hash value as the pattern.

Summary:

Shift s	Text window	t_hash	p_hash	Match	Character Verification
0	a b a c	4	3	No	Skipped
1	b a c a	12	3	No	Skipped
2	a c a b	12	3	No	Skipped
3	c a b a	1	3	No	Skipped
4	a b a b	3	3	Yes	a=a, b=b, a=a, b=b → Match

2.2.4. Complexity

Phase	Time Complexity	Space Complexity
Preprocessing	$O(m)$	$O(1)$
Search (Worst Case)	$O((n - m + 1) * m)$	$O(1)$
Search (Expected)	$O(n + m)$	$O(1)$
Total (Worst Case)	$O(n * m)$	$O(1)$
Total (Expected)	$O(n + m)$	$O(1)$

The worst case arises when every window in T produces a spurious hash match with P , forcing a full character-by-character verification at every position. This can be constructed artificially but is extremely unlikely under a well-chosen prime q .^[2]

2.2.5. Discussion

Rabin-Karp achieves strong average-case performance and excels when multiple patterns are searched simultaneously, as the cost of extending to k patterns is marginal once the rolling hash infrastructure is in place. Its $O(1)$ window-advance step and low constant factor make it efficient in practice for most real-world inputs.^[2]

Its primary weakness lies in the risk of spurious hits – cases where a window's hash matches p_hash but the actual characters do not, forcing an unnecessary verification. If such collisions occur frequently due to a poorly chosen prime modulus, performance degrades to $O(n * m)$. For single-pattern search where worst-case guarantees are required, KMP or Boyer-Moore are generally preferable.^{[1] [2]}

2.3. Knuth-Morris-Pratt (KMP) Algorithm

2.3.1. Introduction and Application

The Knuth-Morris-Pratt algorithm, jointly developed by Donald E. Knuth, James H. Morris, and Vaughan R. Pratt and published in 1977, represents a significant theoretical advancement over the Naïve approach. Its central insight addresses a fundamental inefficiency: when a mismatch occurs at position j within the pattern after successfully matching j characters, the Naïve algorithm discards all information about those matched characters and restarts from the beginning of the pattern at the next shift. KMP eliminates this wasted work by leveraging structural knowledge about the pattern itself.^[3]

During a preprocessing phase, KMP constructs a one-dimensional integer array of length m – referred to in this report as the **failure function array** – where each entry at index j stores the length of the longest proper prefix of the substring $P[0..j]$ that is also a suffix of the same substring. A proper prefix is defined as any prefix that is strictly shorter than the full string. This information encodes the maximum safe backward jump within the pattern upon encountering a mismatch: rather than resetting to position 0, the algorithm can resume comparison from the position indicated by the failure function array, preserving as many of the matched characters as possible.^[3]

KMP is widely used in scenarios requiring strict performance guarantees, most notably in DNA sequence analysis where the small alphabet $\{A, C, G, T\}$ causes repetitive partial matches that severely degrade the Naïve algorithm.^[8] It is also implemented in variants of the Unix **grep** utility and several text editor search engines.^[10]

2.3.2. Pseudocode

COMPUTE-FAILURE-FUNCTION(P):

```
m = length of P
failure[0] = 0
k = 0 // length of current prefix

for q = 1 to (m - 1):
    while k > 0 and P[k] != P[q]:
        k = failure[k - 1]
    if P[k] == P[q]:
        k = k + 1
    failure[q] = k
return failure
```

KMP-MATCHER(T, P):

```
n = length of T
m = length of P
failure = COMPUTE-FAILURE-FUNCTION(P)
q = 0 // number of characters matched so far

for i = 0 to (n - 1):
    while q > 0 and P[q] != T[i]:
        q = failure[q - 1]
    if P[q] == T[i]:
        q = q + 1
    if q == m:
        print "Pattern found at shift" (i - m + 1)
        q = failure[q - 1]
```

2.3.3. Step-by-Step Trace

Using $T = \text{"abacabab"}$ ($n = 8$) and $P = \text{"abab"}$ ($m = 4$).

Preprocessing Phase – Building the Failure Function Array:

The failure function array is computed by examining the pattern P against itself to identify prefix-suffix overlaps:

- **Index 0, $P[0] = \text{'a'}$:** A single character has no proper prefix.

| $\text{failure}[0] = 0$

- **Index 1, P[1] = 'b' (examining P[0..1] = "ab"):** The proper prefixes of "ab" are: {"a"}. The proper suffixes are: {"b"}. No proper prefix equals a proper suffix.

| failure[1] = 0

- **Index 2, P[2] = 'a' (examining P[0..2] = "aba"):** The proper prefixes of "aba" are: {"a", "ab"}. The proper suffixes are: {"a", "ba"}. The longest string appearing in both sets is "a" (length 1).

| failure[2] = 1

- **Index 3, P[3] = 'b' (examining P[0..3] = "abab"):** The proper prefixes of "abab" are: {"a", "ab", "aba"}. The proper suffixes are: {"b", "ab", "bab"}. The longest string appearing in both sets is "ab" (length 2).

| failure[3] = 2

Search Phase – Matching Against Text T:

The variable q tracks how many characters of P have been successfully matched against the current position in T .

- **$i = 0, T[0] = 'a'$:** Compare $P[q] = P[0] = 'a'$ with $T[0] = 'a'$. → Match. q advances from 0 to 1.

| $q = 1$

- **$i = 1, T[1] = 'b'$:** Compare $P[q] = P[1] = 'b'$ with $T[1] = 'b'$. → Match. q advances from 1 to 2.

| $q = 2$

- **$i = 2, T[2] = 'a'$:** Compare $P[q] = P[2] = 'a'$ with $T[2] = 'a'$. → Match. q advances from 2 to 3.

| $q = 3$

- **$i = 3, T[3] = 'c'$:** Compare $P[q] = P[3] = 'b'$ with $T[3] = 'c'$. → Mismatch.

- Consult failure function: $q = \text{failure}[3 - 1] = \text{failure}[2] = 1$. Compare $P[q] = P[1] = 'b'$ with $T[3] = 'c'$. → Still a mismatch.
- Consult failure function: $q = \text{failure}[1 - 1] = \text{failure}[0] = 0$. Compare $P[q] = P[0] = 'a'$ with $T[3] = 'c'$. → Still a mismatch.
- Since $q = 0$, the while loop exits. q remains 0.

- **Observation:** Unlike the Naïve algorithm, which would restart entirely at shift $s = 1$, KMP correctly determines there is no useful prefix of P that can be preserved, and simply advances i . No shift is wasted on the intermediate positions.

| $q = 0$

- $i = 4, T[4] = 'a'$: Compare $P[q] = P[0] = 'a'$ with $T[4] = 'a'$. $\rightarrow$ Match. q advances from 0 to 1.

| $q = 1$

- $i = 5, T[5] = 'b'$: Compare $P[q] = P[1] = 'b'$ with $T[5] = 'b'$. $\rightarrow$ Match. q advances from 1 to 2.

| $q = 2$

- $i = 6, T[6] = 'a'$: Compare $P[q] = P[2] = 'a'$ with $T[6] = 'a'$. $\rightarrow$ Match. q advances from 2 to 3.

| $q = 3$

- $i = 7, T[7] = 'b'$: Compare $P[3] = 'b'$ with $T[7] = 'b'$. $\rightarrow$ Match. q advances from 3 to 4.

- $q = m = 4$. A **valid match** is reported at **shift $i - m + 1 = 7 - 4 + 1 = 4$** .
- q is then reset to $\text{failure}[4 - 1] = \text{failure}[3] = 2$ for the next potential match if any.

| $q = 4 \rightarrow$ Match at $s = 4 \rightarrow q$ reset to 2

Summary:

i	$T[i]$	q (before)	Comparison	Action	q (after)
0	a	0	$P[0]='a'$ vs $'a'$	Match	1
1	b	1	$P[1]='b'$ vs $'b'$	Match	2
2	a	2	$P[1]='b'$ vs $'b'$	Match	3
3	c	3	$P[3]='b'$ vs $'c'$	Mismatch; $q=\text{failure}[2]=1$; $P[1]='b'$ vs $'c'$; $q=\text{failure}[0]=0$; $P[0]='a'$ vs $'c'$; q stays 0	0
4	a	0	$P[0]='a'$ vs $'a'$	Match	1
5	b	1	$P[1]='b'$ vs $'b'$	Match	2
6	a	2	$P[2]='a'$ vs $'a'$	Match	3
7	b	3	$P[3]='b'$ vs $'b'$	Match; $q=4=m \rightarrow$ Match at $s=4$; $q=\text{failure}[3]=2$	2

2.3.4. Complexity

Phase	Time Complexity	Space Complexity
Preprocessing	$O(m)$	$O(m)$
Search	$O(n)$	$O(1)$
Total	$O(n + m)$	$O(m)$

The $O(m)$ space requirement arises from storing the failure function array. The total number of character comparisons across both phases never exceeds $2(n + m)$, guaranteeing the $O(n + m)$ bound regardless of the alphabet size or input structure.^[3]

2.3.5. Discussion

KMP is the algorithm of choice when an unconditional worst-case performance guarantee is required. Its $O(n + m)$ bound is tight and holds across all possible inputs, making it suitable for safety-critical or real-time systems where unpredictable performance spikes are unacceptable. It is especially advantageous over small alphabets, where long partial matches are common and the failure function array yields the greatest savings.^[3]

On large-alphabet text, however, KMP's advantage diminishes. Mismatches tend to occur after the first or second comparison, meaning the failure function is rarely consulted and the linear $O(n)$ search phase provides no practical improvement over the Naïve algorithm. In such settings, Boyer-Moore's ability to skip multiple characters per step results in noticeably faster average-case performance.^{[1] [8]}

2.4. Boyer-Moore Algorithm

2.4.1. Introduction and Application

The Boyer-Moore algorithm, introduced by Robert S. Boyer and J Strother Moore in 1977, is widely regarded as the most efficient string-matching algorithm for practical applications on natural language text. Unlike all previously discussed algorithms, Boyer-Moore scans the characters of the pattern **from right to left** rather than from left to right. This seemingly minor change, combined with two powerful shift heuristics, enables the algorithm to skip over large portions of the text with each mismatch, achieving sub-linear average-case behaviour – meaning it inspects fewer characters than the total length of the text in many practical scenarios.^[4]

Boyer-Moore employs two complementary shift rules, applied simultaneously at each mismatch. The **Bad Character Rule** shifts the pattern so that the mismatching text character aligns with its rightmost occurrence in P , or skips past it entirely if it is absent. The **Good Suffix Rule** shifts the pattern based on the rightmost re-occurrence of the already-matched suffix within P . At each

mismatch, the larger of the two shifts is taken, allowing the algorithm to bypass large portions of the text and achieve an average-case complexity of $O(n/m)$ on large-alphabet text.^{[4] [12]}

Boyer-Moore is the algorithm underlying the GNU grep utility and is the default search engine in many text editors and word processors.^[7] Its sub-linear behaviour makes it the preferred choice for large-scale applications such as file content indexing, antivirus signature scanning, and network intrusion detection, where the text may be hundreds of megabytes in length.^[7]

2.4.2. Pseudocode

```
BUILD-BAD-CHARACTER-TABLE(P, alphabet_size):  
    bad_char = array of size alphabet_size, initialized to -1  
  
    for i = 0 to (length of P - 1):  
        bad_char[P[i]] = i  
  
    return bad_char
```

```
BUILD-SUFFIX-ARRAY(P):  
    m = length of P  
    suffix = array of size m, initialized to 0  
    suffix[m - 1] = m  
    g = m - 1  
    f = 0  
  
    for i = m - 2 downto 0:  
        if i > g and suffix[i + m - 1 - f] < i - g:  
            suffix[i] = suffix[i + m - 1 - f]  
        else:  
            if i < g:  
                g = i  
            f = i  
  
            while g >= 0 and P[g] == P[g + m - 1 - f]:  
                g = g - 1  
  
            suffix[i] = f - g  
  
    return suffix
```

BUILD-GOOD-SUFFIX-TABLE(P):

```
m = length of P
suffix = BUILD-SUFFIX-ARRAY(P)
good_suffix = array of size (m + 1), initialized to m

for i = m - 1 downto 0:
    if suffix[i] == i + 1:
        for j = 0 to (m - 2 - i):
            if good_suffix[j] == m:
                good_suffix[j] = m - 1 - i

for i = 0 to (m - 2):
    good_suffix[m - 1 - suffix[i]] = m - 1 - i

return good_suffix
```

BOYER-MOORE-MATCHER(T, P):

```
n = length of T
m = length of P
bad_char = BUILD-BAD-CHARACTER-TABLE(P)
good_suffix = BUILD-GOOD-SUFFIX-TABLE(P)
s = 0

while s <= (n - m):
    j = m - 1
    while j >= 0 and P[j] == T[s + j]:
        j = j - 1

    if j < 0:
        print "Pattern found at shift s"
        s = s + good_suffix[0]
    else:
        bc_shift = j - bad_char[T[s + j]]
        gs_shift = good_suffix[j]
        s = s + max(bc_shift, gs_shift)
```


2.4.3. Step-by-Step Trace

Using $T = \text{"abacabab"} (n = 8)$ and $P = \text{"abab"} (m = 4)$.

Preprocessing Phase – Building the Bad Character Table:

Scanning P left to right, recording the rightmost index of each character (later occurrences overwrite earlier ones):

- $P[0] = 'a' \rightarrow \text{bad_char}['a'] = 0$
- $P[1] = 'b' \rightarrow \text{bad_char}['b'] = 1$
- $P[2] = 'a' \rightarrow \text{bad_char}['a'] = 2$ (overwrites)
- $P[3] = 'b' \rightarrow \text{bad_char}['b'] = 3$ (overwrites)

Preprocessing Phase – Building the Suffix Array:

The suffix array records for each position i the length of the longest substring of P ending at position i that is also a suffix of the full pattern P .

$P = \text{"abab"} (P[0]='a', P[1]='b', P[2]='a', P[3]='b')$.

- The full string "abab" ending at $i=3$ is trivially a suffix of itself (length 4).

| `suffix[3] = 4`

- The substrings ending at $i=2$ are "a" and "ba" and "aba". The suffixes of P are "b", "ab", "bab", "abab". None match.

| `suffix[2] = 0`

- The substrings ending at $i=1$ are "b" and "ab". Both "b" (length 1) and "ab" (length 2) are suffixes of P . The longest is "ab".

| `suffix[1] = 2`

- The only substring ending at $i=0$ is "a". The length-1 suffix of P is "b". No match.

| `suffix[0] = 0`

Preprocessing Phase – Building the Suffix Table:

The good suffix table has size $m + 1 = 5$, initialised entirely to $m = 4$.

| `good_suffix = [4, 4, 4, 4, 4]`

Case 1 – No copy of the matched suffix exists elsewhere in P . For each i where `suffix[i]` equals $i + 1$, fill shift values for all positions to the left of i :

- $i = 3$: `suffix[3] = 4 = 3 + 1 = 4. The range  $j = 0$  to  $(m - 2 - i) = (4 - 2 - 3) = -1$  is empty. No update.`
- $i = 2$: `suffix[2] = 0  $\neq$  3`. Skip.
- $i = 1$: `suffix[1] = 2 = 1 + 1 = 2`. The range $j = 0$ to $(m - 2 - i) = 1$:
 - $j = 0$: `good_suffix[0] = 4` (unchanged) $\rightarrow$ set `good_suffix[0] =  $m - 1 - i = 2$`
 - $j = 1$: `good_suffix[1] = 4` (unchanged) $\rightarrow$ set `good_suffix[1] =  $m - 1 - i = 2$`
- $i = 0$: `suffix[0] = 0  $\neq$  1`. Skip.

| `good_suffix = [2, 2, 4, 4, 4]` // after case 1

Case 2 – A copy of the matched suffix exists at another position in P . For each i from 0 to $m - 2$, set `good_suffix[m - 1 - suffix[i]] =  $m - 1 - i$` :

- $i = 0$: `suffix[0] = 0`. `good_suffix[m - 1 - 0] = good_suffix[3] =  $m - 1 - 0 = 3$`
- $i = 1$: `suffix[1] = 2`. `good_suffix[m - 1 - 2] = good_suffix[1] =  $m - 1 - 1 = 2$` (no change)
- $i = 2$: `suffix[2] = 0`. `good_suffix[m - 1 - 0] = good_suffix[3] =  $m - 1 - 2 = 1$` (overwrites)

| `good_suffix = [2, 2, 4, 1, 4]` // after case 2

Interpretation of the table entries:

- `good_suffix[0] = 2` – After a full match ($j < 0$), shift by 2. The prefix "ab" of P matches the suffix "ab" of P , so the next alignment reuses those two characters.

- `good_suffix[1] = 2` – Mismatch at $j=1$ after matching `P[2..3]="ab"`. The suffix "ab" reappears starting at `P[0]`. Shift by 2 to align it.

- `good_suffix[2] = 4` – Mismatch at $j=2$ after matching `P[3]="b"`. No other occurrence of "b" as a suffix-aligned substring exists. Shift by the full pattern length 4.

- $\text{good_suffix}[3] = 1$ – Mismatch at $j=3$ (no characters matched). Minimum shift of 1 applied; the bad character rule will dominate in practice.

Search Phase – Searching Text T:

- **Shift $s = 0$** – Aligning P at $T[0..3] = \text{"abac"}$, comparing right to left:

- $j = 3$: $P[3] = \text{'b'}$ vs $T[3] = \text{'c'}$ → **Mismatch**. Applying both rules:
 - Bad Character Rule: $\text{bad_char}[\text{'c'}] = -1$. $\text{bc_shift} = j - \text{bad_char}[\text{'c'}] = 3 - (-1) = 4$.
 - Good Suffix Rule: $\text{gs_shift} = \text{good_suffix}[3] = 1$.
 - $\text{Shift} = \max(4, 1) = 4$. New $s = 0 + 4 = 4$

- **Shift $s = 4$** – Aligning P at $T[4..7] = \text{"abab"}$, comparing right to left:

- $j = 3$: $P[3] = \text{'b'}$ vs $T[7] = \text{'b'}$ → Match.
- $j = 2$: $P[2] = \text{'a'}$ vs $T[6] = \text{'a'}$ → Match.
- $j = 1$: $P[1] = \text{'b'}$ vs $T[5] = \text{'b'}$ → Match.
- $j = 0$: $P[0] = \text{'a'}$ vs $T[4] = \text{'a'}$ → Match.

j has reached -1 . **A valid match is reported at shift $s = 4$** . The algorithm then applies $\text{good_suffix}[0] = 2$, yielding $s = 4 + 2 = 6 > n - m = 4$. The while loop terminates.

Summary:

Shift s	Window	Mismatch at	bc shift	gs shift	Shift Taken	Outcome
0	a b a c	$j=3$ ('c' vs 'b')	4	1	$\max(4,1) = 4$	Advance to $s=4$
4	a b a b	None	–	–	$\text{good_suffix}[0]=2$	Match at $s=4$

2.4.4. Complexity

Phase	Time Complexity	Space Complexity
Bad character table	$O(m + \Sigma)$	$O(\Sigma)$
Suffix array	$O(m)$	$O(m)$
Good suffix table	$O(m)$	$O(m)$
Search (Best / Average Case)	$O(n/m)$	$O(1)$
Search (Worst Case)	$O(n + m)$	$O(1)$
Total (Best / Average Case)	$O(n/m + m + \Sigma)$	$O(m + \Sigma)$
Total (Worst Case)	$O(n + m + \Sigma)$	$O(m + \Sigma)$

The inclusion of the Good Suffix Rule is what eliminates the quadratic worst case present in Bad Character-only implementations, reducing it from $O(n * m)$ to a guaranteed $O(n + m)$. The sub-linear average-case behaviour of $O(n/m)$ is preserved regardless, as it arises from the large skips produced by both heuristics on typical inputs.^{[4] [6] [12]}

2.4.5. Discussion

Boyer-Moore consistently delivers the best practical performance when compared with the previous algorithms, particularly on large alphabets with moderately long patterns. When the mismatching text character does not appear in P , the entire pattern length m is skipped in a single step, allowing the algorithm to inspect only a small fraction of the total text characters. This sub-linear behaviour becomes increasingly pronounced as m grows larger.^[4]

Its primary disadvantage is the significant implementation complexity of the full algorithm. The Good Suffix Rule in particular requires careful construction of an auxiliary table and is a common source of implementation errors.^[6] Additionally, Boyer-Moore's advantage diminishes substantially on small alphabets, such as binary or DNA sequences, where nearly every text character appears somewhere in the pattern, reducing bad-character shifts to near zero and eliminating the sub-linear benefit.^[4]

2.5. Aho-Corasick Algorithm

2.5.1. Introduction and Application

The Aho-Corasick algorithm, introduced by Alfred V. Aho and Margaret J. Corasick in 1975, is a multi-pattern string-matching algorithm that extends the principles of KMP to handle an arbitrary number of patterns simultaneously.^[5] Rather than searching for each keyword independently, the algorithm constructs a finite-state automaton^[9] – commonly referred to as the **Aho-Corasick automaton** – from the complete set of K patterns during preprocessing. This automaton consists of three components: a **goto function** that defines forward transitions between states upon matching a character, a **failure function** that defines fallback transitions upon a mismatch (analogous to the failure function array in KMP), and an **output function** that records which patterns are matched at each state. Once constructed, the automaton processes the text T in a single left-to-right pass, transitioning between states one character at a time and reporting all pattern matches encountered along the way.^[5]

The Aho-Corasick algorithm is the standard solution for multi-pattern search in performance-critical applications. It is the underlying algorithm of the Unix `fgrep` command, which searches for multiple fixed strings simultaneously. It is also extensively used in network intrusion detection systems such as Snort, where incoming packets must be scanned against thousands of known attack

signatures in real time.^[11] In the context of this project, Aho-Corasick is particularly well-suited for scenario where a large number of keywords K must be located within the grid, as all K patterns are searched in a single pass rather than K independent passes.^[5]

2.5.2. Pseudocode

```
BUILD-GOTO-FUNCTION(patterns):
    state_count = 0
    goto = empty map of (state, char) -> state
    output = empty map of state -> set of patterns

    for each pattern p in patterns:
        current = 0
        for each character c in p:
            if (current, c) not in goto:
                state_count = state_count + 1
                goto[(current, c)] = state_count
            current = goto[(current, c)]
        output[current] = output[current] UNION {p}

    for each character c in alphabet:
        if (0, c) not in goto:
            goto[(0, c)] = 0

    return goto, output, state_count
```

```
BUILD-FAILURE-FUNCTION(goto, output, state_count):
    failure = array of size (state_count + 1), initialized to 0
    queue = empty queue

    // Depth-1 states all fail back to root
    for each character c in alphabet:
        s = goto[(0, c)]
        if s != 0:
            failure[s] = 0
            enqueue s into queue

    // BFS to compute failure links for deeper states
    while queue is not empty:
        r = dequeue from queue
        for each character c in alphabet:
            if (r, c) in goto:
                s = goto[(r, c)]
                enqueue s into queue
                state = failure[r]
                while state != 0 and (state, c) not in goto:
                    state = failure[state]
                failure[s] = goto[(state, c)]
                if failure[s] == s:
                    failure[s] = 0

            output[s] = output[s] UNION output[failure[s]]

    return failure, output
```

```
AHO-CORASICK-SEARCH(T, goto, failure, output):
    current = 0    // begin at root state
    for i = 0 to (length of T - 1):
        while current != 0 and (current, T[i]) not in goto:
            current = failure[current]    // follow failure link
        if (current, T[i]) in goto:
            current = goto[(current, T[i])]
        if output[current] is not empty:
            for each pattern p in output[current]:
                print "Pattern p found ending at position i"
```

2.5.3. Step-by-Step Trace

The following trace uses two patterns $P1 = "ab"$ and $P2 = "bc"$, and text $T = "abcb"$ ($n = 5$).

Preprocessing Phase – Building Goto Function:

Each pattern is inserted into a trie beginning from the root (state 0), with a new state created for each character that does not yet have a defined transition.

- Inserting $P1 = "ab"$:

- State 0 $\xrightarrow{'a'}$ State 1 (new)
- State 1 $\xrightarrow{'b'}$ State 2 (new)
- $output[2] = \{"ab"\}$

- Inserting $P2 = "bc"$:

- State 0 $\xrightarrow{'b'}$ State 3 (new)
- State 3 $\xrightarrow{'c'}$ State 4 (new)
- $output[4] = \{"bc"\}$

All remaining undefined transitions from the root (state 0) are set to loop back to state 0.

Root (0) $\xrightarrow{'a'}$ (1)	Root (0) $\xrightarrow{'b'}$ (3)	$output[2] = \{"ab"\}$
Root (0) $\xrightarrow{'b'}$ (3)	Root (0) $\xrightarrow{'c'}$ (4)	$output[4] = \{"bc"\}$

Preprocessing Phase – Building Failure Function:

Failure links are computed via breadth-first search (BFS) starting from all depth-1 states. Each failure link points to the state representing the longest proper suffix of the current state's string that is also a valid prefix path from the root in the automaton.

- **Depth-1 states** are initialised with failure links pointing to root (state 0):

- $failure[1] = 0$ (state 1 represents "a"; no proper suffix of "a" is a recognised prefix in the automaton other than the empty string)
- $failure[3] = 0$ (state 3 represents "b"; similarly falls back to root)
- Both states 1 and 3 are enqueued for BFS processing.

- Processing **State 1** (representing "a"):

- Transition on 'b' leads to state 2 (representing "ab").
- To compute $failure[2]$: follow $failure[1] = 0$, then check $goto[(0, 'b')] = 3$. State 3 exists and is not state 2 itself.
- $failure[2] = 3$.
- Propagate output: $output[2] = output[2] \cup output[3] = \{"ab"\} \cup \{\} = \{"ab"\}$.

- Processing **State 3** (representing "b"):

- Transition on 'c' leads to state 4 (representing "bc").
- To compute failure[4]: follow failure[3] = 0, then check goto[(0, 'c')] = 0 (undefined, loops to root).
- failure[4] = 0.
- Propagate output: output[4] = output[4] $\cup$ output[0] = {"bc"} $\cup$ {} = {"bc"}.

The failure link from state 2 to state 3 is particularly significant: after fully matching "ab", the automaton recognises that the suffix "b" of "ab" is also the beginning of a valid path toward "bc" in the automaton. Rather than returning to root, it jumps directly to state 3, preserving partial progress toward the second pattern without re-examining any character.

Search Phase – Searching Text T:

Starting at state 0, the automaton processes T one character at a time, following goto transitions on matches and failure links on mismatches.

- **i = 0, T[0] = 'a':**

- Current state is 0.
- Check goto[(0, 'a')] = 1 – transition is defined. Move to state 1.
- output[1] is empty.
- No match reported.

| current = 1

- **i = 1, T[1] = 'b':**

- Current state is 1.
- Check goto[(1, 'b')] = 2 – transition is defined. Move to state 2.
- output[2] = {"ab"} is not empty.
- **Pattern "ab" is reported, ending at i = 1, starting at i = 0.**

| current = 2

- **i = 2, T[2] = 'c':**

- Current state is 2.
- Check goto[(2, 'c')] – transition is not defined.
 - Follow failure link: failure[2] = 3. Move to state 3.
 - Current state is 3.
 - Check goto[(3, 'c')] = 4 – transition is defined. Move to state 4.
 - output[4] = {"bc"} is not empty.
 - **Pattern "bc" is reported, ending at i = 2, starting at i = 1.**
- **Observation:** The automaton did not return to the root upon the mismatch at state 2. Instead, the failure link carried it directly to state 3, which represents the suffix "b" of the previously matched string "ab". From state 3, the transition on 'c' immediately completes a match for "bc", detecting an overlapping pattern occurrence without re-examining any prior character.

| current = 4

- **i = 3, T[3] = 'a':**

- Current state is 4.
- Check goto[(4, 'a')] – transition is not defined.
 - Follow failure link: failure[4] = 0. Move to state 0.
 - Current state is 0.
 - Check goto[(0, 'a')] = 1 – transition is defined. Move to state 1.
 - output[1] is empty. No match reported.

| current = 1

- **i = 4, T[4] = 'b':**

- Current state is 1.
- Check goto[(1, 'b')] = 2 – transition is defined. Move to state 2.
- output[2] = {"ab"} is not empty.
- **Pattern "ab" is reported, ending at i = 4, starting at i = 3.**

| current = 2

Summary:

i	T[i]	State (before)	Transition	State (after)	Match Reported
0	a	0	goto[(0,'a')] = 1	1	None
1	b	1	goto[(1,'b')] = 2	2	output[2] = "ab". Ending at i=1. Starting at i=0.
2	c	2	(2,'c') not defined → follow failure[2]=3; goto[(3,'c')]=4	4	output[4] = "bc". Ending at i=2. Starting at i=1.
3	a	4	(4,'a') not defined → follow failure[4]=0; goto[(0,'a')]=1	1	None
4	b	1	goto[(1,'b')] = 2	2	output[2] = "ab". Ending at i=4. Starting at i=3.

2.5.4. Complexity

Phase	Time Complexity	Space Complexity
Goto function	$O(M)$	$O(M * \Sigma)$
Failure function	$O(M * \Sigma)$	$O(M)$
Output function	$O(M)$	$O(M)$
Search	$O(n)$	$O(1)$
Total	$O(n + M * \Sigma)$	$O(M * \Sigma)$

M denotes the sum of the lengths of all K patterns ($M = m_1 + m_2 + \dots + m_K$). Once the automaton is constructed, the search phase runs in strict $O(n)$ time regardless of K or M — a property unique among all algorithms presented in this report. The preprocessing cost $O(M * |\Sigma|)$ is incurred only once and is shared across all K patterns, making the algorithm increasingly cost-efficient as K grows larger.^[5]

2.5.5. Discussion

Aho-Corasick is the most powerful algorithm in this report for the multi-pattern setting. Because all K patterns are encoded into a single automaton, the text T is traversed exactly once during the search phase regardless of K , giving a search cost entirely independent of the number of keywords. This makes it strictly superior to running KMP or Boyer-Moore independently K times, which would incur a search cost of $O(n * K)$, and it avoids the hash collision risk inherent in Rabin-Karp's multi-pattern extension. Aho-Corasick offers the greatest efficiency advantage in scenario where K is large and the cost of K independent passes becomes significant.^{[5] [11]}

Its primary disadvantage is the substantial memory requirement during preprocessing. The goto function in its array-based representation allocates a table of size $(\text{total states}) \times |\Sigma|$, which for a large dictionary of long patterns can consume significant memory, particularly over a 26-character alphabet. ^[11] Additionally, the automaton construction is more complex to implement correctly than any of the four core algorithms, particularly the failure function BFS traversal and the propagation of the output function through failure links. For scenarios involving only a single pattern or a very small K , simpler algorithms such as KMP or Boyer-Moore remain more appropriate choices. ^[5]

3. Experiment

3.1. Context

The experiments presented in this section evaluate the performance of the five string-matching algorithms – Naïve, Rabin-Karp, KMP, Boyer-Moore and Aho-Corasick – within the context of a two-dimensional character grid, as specified by the project requirements. Unlike the one-dimensional string formulation described in Section 2, the experimental setting introduces a structured grid of **R rows** and **C columns**, where each cell contains a single lowercase English character. This two-dimensional structure necessitates a straightforward adaptation of each algorithm: rather than operating on a single linear text T , the search procedure is applied independently to each extracted row (for horizontal matching, left-to-right) and each extracted column (for vertical matching, top-to-bottom). Each row and column is treated as a separate one-dimensional string of length C and R respectively, and the selected algorithm is invoked on each such string in turn. **The core logic of each algorithm therefore remains unchanged** from its theoretical formulation; only the manner in which the text strings are derived from the grid differs.

To obtain meaningful and reproducible performance measurements, two experimental scenarios are conducted as outlined in the project specification. In Scenario 1, the number of keywords K is held constant at a small value while the grid dimensions are progressively increased from 10×10 to 500×500 , allowing the relationship between **grid size** and algorithm performance to be observed in isolation. In Scenario 2, the grid size is fixed at a moderate 100×100 configuration while the number of keywords K is varied across 10, 50, and 100 values, in order to examine how each algorithm scales with an **increasing search workload**. For each configuration, two performance metrics are recorded: the total number of character comparisons performed across all row and column searches, and the execution time of the search phase measured in milliseconds, excluding all input parsing and grid construction overhead. The results of both scenarios are reported in tabular format in the following subsection along with visualizations for each.

3.2. Results

3.2.1. Scenario 1

Grid size	Num. Keywords	Brute Force	Rabin-Karp	KMP	Boyer-Moore	Aho-Corasick
10 x 10	5	0.02892	0.04988	0.03992	0.0338	0.24104
100 x 100	5	2.15266	2.36894	1.69144	0.76684	3.13544
500 x 500	5	37.12712	35.76334	40.16316	14.81804	33.6526

Table 3.2.1.a Scenario 1

Scenario 1 (Fixed key number = 5)

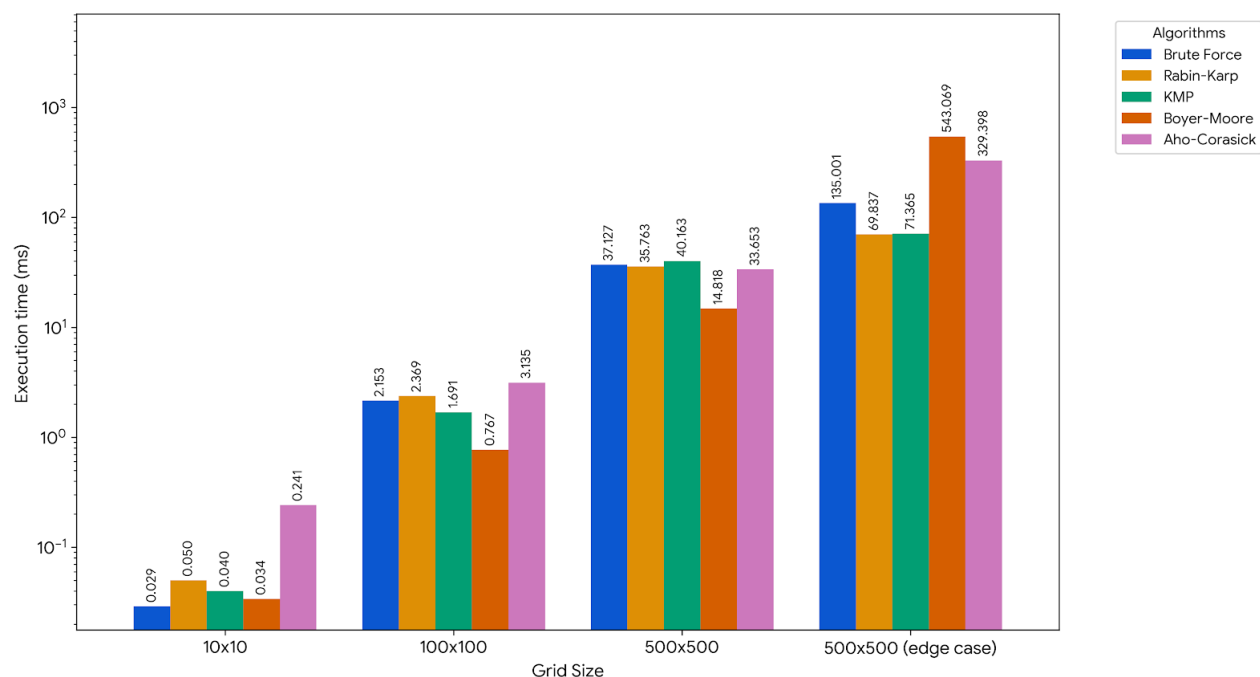


Chart 3.2.1.b Scenario 1

The observed growth rate generally aligns with the Big-O complexity analyzed earlier, but only at a high level. As the grid size increases from 10×10 to 100×100 and 500×500, the execution time of all algorithms increases significantly, which is consistent with their theoretical time complexities. It is important to note that the **normal case results are averaged over 5 trials with randomized input**, which helps reduce variance and provides a more reliable estimate of typical performance. However, the growth is not perfectly proportional to the asymptotic analysis, as constant factors and implementation details still have a strong influence on actual runtime.

On large datasets, Boyer–Moore clearly performs the best. At the 500×500 grid size, it has the lowest execution time compared to all other algorithms, significantly outperforming Brute Force, KMP, Rabin–Karp, and Aho–Corasick. This is mainly because Boyer–Moore can skip large portions of the text when mismatches occur, leading to sublinear behavior in many practical cases. In contrast, algorithms like KMP and Rabin–Karp still process most of the input sequentially, which results in higher execution times as the dataset grows.

There are cases where the Brute Force approach is faster than the optimized algorithms, particularly for very small datasets such as the 10×10 grid. In this case, Brute Force achieves the lowest execution time among all methods. This happens because it has no preprocessing overhead, while algorithms like KMP, Rabin–Karp, and Aho–Corasick require additional setup steps such as building prefix tables, computing hashes, or constructing automata. For small inputs, this overhead dominates the total runtime, making the simpler approach more efficient.

The edge case results should be interpreted more cautiously, as they are **obtained from only a single run rather than averaged over multiple trials**. This means they may be more sensitive to specific input characteristics and can exhibit higher variability. For example, algorithms like Rabin–Karp may suffer from hash collisions, and Aho–Corasick may incur more state transitions depending on the input structure, leading to noticeably higher execution times in the edge case.

Overall, these results can be explained by the difference between theoretical complexity and practical performance. While Big-O notation describes how algorithms scale with input size, it does not account for constant factors, preprocessing costs, or input distribution. Boyer–Moore benefits from efficient skipping strategies, making it very fast in practice. Brute Force performs well on small inputs due to its simplicity, and Aho–Corasick does not perform optimally here because it is designed for handling a large number of patterns, whereas only five keywords are used. This demonstrates that algorithm performance depends not only on theory but also on experimental setup and real-world conditions.

3.2.2. Scenario 2

Grid size	Num. Keywords	Brute Force	Rabin-Karp	KMP	Boyer-Moore	Aho-Corasick
100 x 100	10	4.09594	4.83562	4.0445	1.34664	3.22626
100 x 100	50	17.83834	15.94862	16.9269	7.29008	3.85056
100 x 100	100	39.39746	32.90784	34.75334	14.54432	4.85602

Table 3.2.2.a Scenario 2

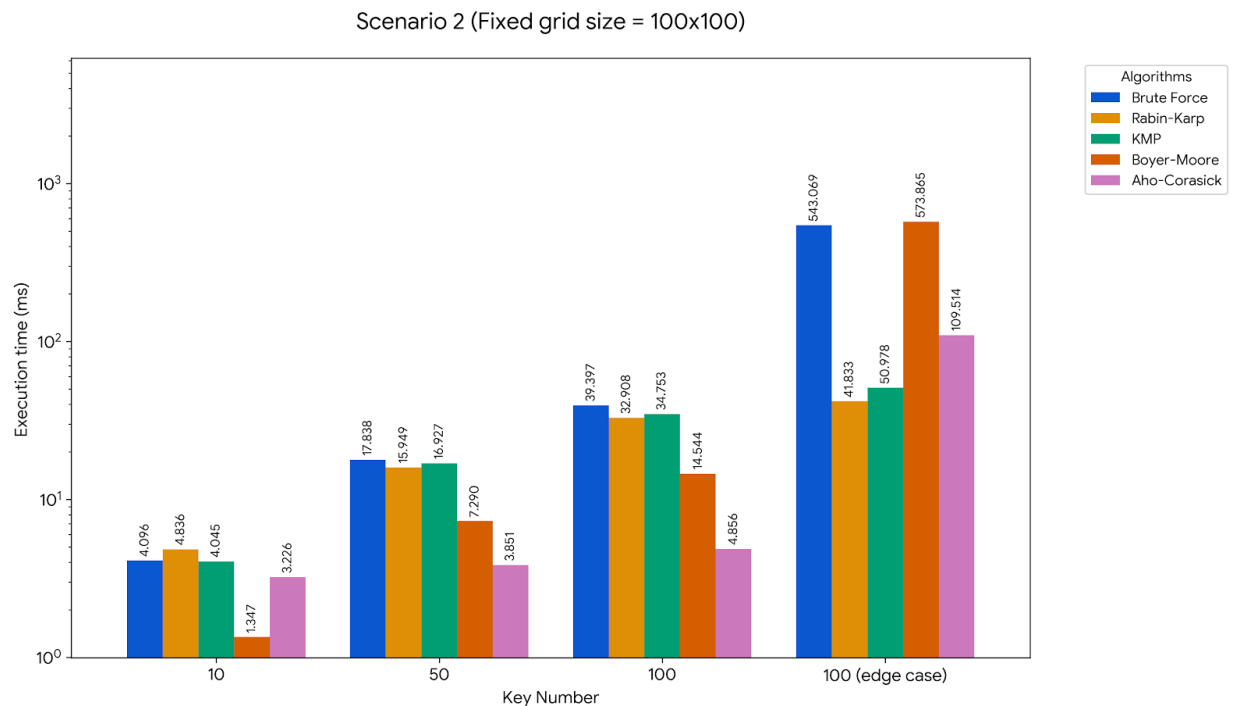


Table 3.2.2.b Scenario 2

The results in Scenario 2 generally follow the expected trend from Big-O complexity, but again only at a high level. Since the grid size is fixed at 100×100, the main factor affecting performance is the **number of keywords**. As the number of patterns increases from 10 to 50 and 100, the execution time of all algorithms also increases. The **normal case values are averaged over 5 trials with randomized input**, which helps produce smoother and more reliable results across different keyword sizes. This makes the upward trend more consistent and easier to interpret. However, the increase is not perfectly linear or identical across algorithms due to differences in internal processing and overhead.

In terms of performance, **Aho–Corasick performs best as the number of keywords becomes large**. At 100 keywords, it has the lowest execution time among all algorithms, significantly outperforming Brute Force, Rabin–Karp, KMP, and even Boyer–Moore. This is expected because Aho–Corasick is specifically designed for multi-pattern matching, allowing it to process all keywords simultaneously using a trie-based automaton. In contrast, the other algorithms typically handle each pattern separately, leading to a rapid increase in total runtime as the number of keywords grows.

Boyer–Moore performs very well for a smaller number of keywords (e.g., 10 and 50), where it achieves the fastest or near-fastest times due to its efficient skipping strategy. However, as the number of keywords increases to 100, its performance degrades significantly. This happens because Boyer–Moore must be applied repeatedly for each pattern, so its advantage in single-pattern matching diminishes when scaling to many patterns.

The edge case results should be interpreted more cautiously because they are **based on a single run rather than an average of multiple trials**. As a result, these values are more sensitive to specific input configurations and may show larger deviations. For example, in the edge case with 100 keywords, both Brute Force and Boyer–Moore exhibit a dramatic increase in execution time, suggesting that the input may have caused more frequent matches or reduced opportunities for skipping. Similarly, Rabin–Karp may experience more hash collisions, and Aho–Corasick may traverse more states depending on the structure of the input.

Overall, these results highlight that algorithm performance depends heavily on the number of patterns in addition to input size. While theoretical complexity provides a useful guideline, practical performance is influenced by preprocessing overhead, the ability to reuse computations across patterns, and the structure of the input data. In this scenario, Aho–Corasick emerges as the most scalable solution for large keyword sets, while Boyer–Moore remains efficient for smaller sets, and Brute Force becomes increasingly inefficient as the number of keywords grows.

4. References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009, ch. 32, pp. 985–1013.
- [2] R. M. Karp and M. O. Rabin, "Efficient randomized pattern-matching algorithms," *IBM Journal of Research and Development*, vol. 31, no. 2, pp. 249–260, Mar. 1987. doi: 10.1147/rd.312.0249
- [3] D. E. Knuth, J. H. Morris, Jr., and V. R. Pratt, "Fast pattern matching in strings," *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, Jun. 1977. doi: 10.1137/0206024
- [4] R. S. Boyer and J. S. Moore, "A fast string searching algorithm," *Communications of the ACM*, vol. 20, no. 10, pp. 762–772, Oct. 1977. doi: 10.1145/359842.359859
- [5] A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, Jun. 1975. doi: 10.1145/360825.360855
- [6] C. Charras and T. Lecroq, *Handbook of Exact String Matching Algorithms*. London, UK: King's College London Publications, 2004. [Online]. Available: <https://www-igm.univ-mlv.fr/~lecroq/string/>
- [7] Z. D. Syed, S. Stakhonova, and A. A. Ghorbani, "Application of Boyer-Moore algorithm in intrusion detection systems," in *Proc. 22nd Annual Computer Security Applications Conference (ACSAC)*, Miami, FL, USA, Dec. 2006, pp. 125–134. doi: 10.1109/ACSAC.2006.24
- [8] D. Gusfield, *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*. Cambridge, UK: Cambridge University Press, 1997, ch. 2–4, pp. 11–85.
- [9] V. M. Glushkov, "The abstract theory of automata," *Russian Mathematical Surveys*, vol. 16, no. 5, pp. 1–53, 1961. doi: 10.1070/RM1961v016n05ABEH004112
- [10] cp-algorithms, "String – Prefix Function and KMP Algorithm," Competitive Programming Algorithms, 2024. [Online]. Available: <https://cp-algorithms.com/string/prefix-function.html> [Accessed: Mar. 2026].
- [11] cp-algorithms, "String – Aho-Corasick Algorithm," Competitive Programming Algorithms, 2024. [Online]. Available: https://cp-algorithms.com/string/aho_corasick.html [Accessed: Mar. 2026].
- [12] GeeksforGeeks, "Boyer-Moore Algorithm for Pattern Searching," GeeksforGeeks, 2024. [Online]. Available: <https://www.geeksforgeeks.org/boyer-moore-algorithm-for-pattern-searching/> [Accessed: Mar. 2026].

5. Acknowledgment

Support was provided by students from class 25C10 in providing ideas and explaining the foundation of the problem, among others. In addition, AI tools, such as Gemini and Claude, assisted with recommending references/citations, proofreading and improving the report (format, language and grammar) as well as providing example for Tracing but not generating any core contents or codebase.